

PYTHON PROGRAMMING

1. What is Python?

- Python is a high-level, interpreted, and object-oriented programming language.
- It was created by Guido van Rossum and released in 1991.
- Python is easy to learn because of its simple and readable syntax.
- It is widely used in Web Development, AI, Data Science, Machine Learning, Cyber Security, Automation, Cloud Computing and Software Development.

2. Why Python?

- Easy to learn and write.
- Simple and readable syntax.
- Free and Open Source.
- Large community support.
- Huge collection of libraries.
- Platform independent.
- Supports Object-Oriented Programming.
- Used by top companies like Google, Netflix, Microsoft, Instagram, Spotify.

PYTHON PROGRAMMING

3. Features of Python

Feature	Description
Easy Syntax	Python has simple and readable syntax which is easy to learn.
Interpreted Language	Python is an interpreted language, so no compilation is required.
Object-Oriented	It supports OOP concepts like class, objects, inheritance, etc.
High-Level Language	Python is a high-level language, close to human language.
Open Source	Python is free to use and open source.
Portable	Python is platform independent. It runs on Windows, Linux and Mac.
Large Standard Library	Python has a large collection of built-in modules and libraries.
Dynamic Typing	Variables are not required to declare the data type explicitly.
Automatic Memory Management	Python has garbage collection to manage memory automatically.

4. Applications of Python

- Web Development
- Artificial Intelligence (AI)
- Machine Learning
- Data Science
- Cyber Security
- Ethical Hacking
- Automation / Scripting
- Data Analysis
- Game Development
- Desktop Applications
- Cloud Computing

Quick Note :

→ Python = Simple + Powerful + Versatile



PYTHON PROGRAMMING

5. History of Python

- Python was created by Guido van Rossum.
- He started working on Python in December 1989.
- The first version, Python 0.9.0, was released in February 1991.
- Python 2.0 was released in 2000 with many new features.
- Python 3.0 was released in 2008, which is fully new and not backward compatible.
- Python 3.x is the latest and most used version.

Timeline of Python Versions

Version	Release Year
Python 0.9.0	1991
Python 1.0	1994
Python 2.0	2000
Python 2.7	2010
Python 3.0	2008
Python 3.x	Latest (Recommended)

Creator of Python

Guido van Rossum



Born : 31 January 1956
(Netherlands)

6. Advantages of Python

- Easy to learn and use.
- Readable and clean syntax.
- Large community and support.
- Many built-in libraries and modules.
- Platform independent.
- High-level language.
- Supports multiple programming paradigms.
- Increases productivity and reduces development time.

Python is a great choice for beginners as well as professionals.

7. Installation of Python

1. Go to www.python.org
2. Click on the "Downloads" menu.
3. Download the latest version of Python.
4. Run the installer file.
5. Important: Check the option "Add Python to PATH".
6. Click on "Install Now" and wait for installation.
7. Click on "Close" after successful installation.
8. Verify installation by opening Command Prompt and typing:

```
python --version
```

→ It will show the installed Python version.



8. Python IDEs (Integrated Development Environment)

- IDLE - Default IDE with Python installation. Simple and easy to use.
- PyCharm - Powerful IDE with many advanced features.
- VS Code - Lightweight and supports many extensions.
- Jupyter Notebook - Used for data science and machine learning.
- Spyder - Good IDE for scientific programming.

9. Writing and Running Python Program

Steps:

1. Open any IDE (IDLE / VS Code / PyCharm).
2. Create a new file with .py extension (example: hello.py)
3. Write your Python code.
4. Save the file.
5. Run the program.

Example:

```
# This is a simple Python program
print("Hello, World!")
name = "Monika"
print("Welcome.", name)
```

Output:

```
Hello, World!
Welcome, Monika
```

★ Note: Python is a case-sensitive language.
"Python", "python" and "PYTHON" are different.

10. Variables in Python

- A variable is a container used to store data values.
- In Python, we do not need to declare the type of variable. It is automatically taken care by Python.
- The value of a variable can be changed during program execution.
- Variable name must start with a letter (a-z, A-Z) or underscore (_).
- Variable name cannot start with a number.
- No special characters allowed except underscore (_).
- Python is case-sensitive (age, Age and AGE are different).

Example :

Valid Variable Names ✓	Invalid Variable Names ✗
• name • _name • name123 • Name • age_1 • totalMarks	• 1name (starts with number) • name@123 (special character) • total-marks (hyphen not allowed) • my name (space not allowed) • !age (special character)

Rules for Variable Names

1. Must start with a letter or underscore (_)
2. Can contain letters, digits and underscore only
3. Cannot start with a number
4. No special characters allowed (except _)
5. Case-sensitive

11. Declaring and Assigning Variables

- In Python, we can declare a variable and assign a value in one line.

Example :

```
x = 10           # integer variable
name = "Monika"  # string variable
pi = 3.14        # float variable
is_student = True # boolean variable
```

★ Quick Note

We do not use any keyword like int, float, etc. to declare a variable in Python.

12. Multiple Assignment

- We can assign values to multiple variables in a single line.

Examples :

1. Assigning same value

```
a = b = c = 5
print(a, b, c) # 5 5 5
```

2. Assigning different values

```
x, y, z = 10, 20, 30
print(x, y, z) # 10 20 30
```

3. Using multiple assignment

```
p, q = 100, 200
print(p) # 100
print(q) # 200
```

★ Variables make our programs flexible and reusable. ★

13. Data Types in Python

- Data types specify the type of value a variable can hold.
- Python has the following built-in data types.

Data Type	Description	Example	Type
int	Integer (whole number)	10, -5, 0, 100	int
float	Floating point number	3.14, -0.01, 2.0	float
str	String (sequence of characters)	'Hello', "Python"	str
bool	Boolean value (True / False)	True, False	bool
list	Ordered collection (mutable)	[1, 2, 3, 'A']	list
tuple	Ordered collection (immutable)	(1, 2, 3, 'A')	tuple
set	Unordered collection of unique items	{1, 2, 3}	set
dict	Key-value pair collection	{'name': 'Monika', 'age': 21}	dict

Example :

a = 10	# int	e = [1, 2, 3]	# list
b = 3.14	# float	f = (1, 2, 3)	# tuple
c = "Monika"	# str	g = {1, 2, 3}	# set
d = True	# bool	h = {'name': 'Monika', 'age': 21}	# dict

14. Type Conversion in Python

- We can convert one data type into another data type.
- This process is called type casting.

Common Type Conversions :

Function	Description	Example
int()	Converts to integer	int(3.14) → 3
float()	Converts to float	float(10) → 10.0
str()	Converts to string	str(100) → '100'
bool()	Converts to boolean	bool(0) → False

Example :

a = 5	
b = float(a)	# 5.0
c = str(a)	# '5'
d = bool(a)	# True

★ Understanding data types is very important for writing efficient programs. ★

15. Operators in Python

- Operators are used to perform operations on variables and values.
- Python supports the following types of operators.

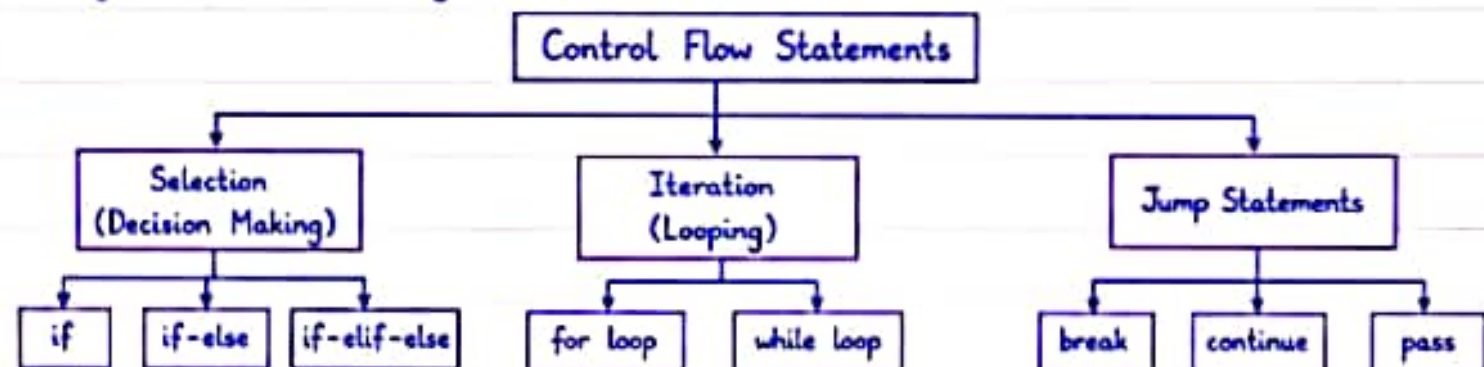
Type	Operators	Description	Example
Arithmetic	+, -, *, /, %, //, **	Performs mathematical operations	a + b, a % b
Comparison	==, !=, >, <, >=, <=	Compares two values	a == b, a >= b
Assignment	=, +=, -=, **=, /=, %=	Assigns values to variables	a = 10, a += 5
Logical	and, or, not	Combines conditional statements	a > 5 and b < 10
Bitwise	&, , ^, ~, <<, >>	Performs bit-level operations	a & b, a << 2
Membership	in, not in	Checks membership in a sequence	"x" in str
Identity	is, is not	Checks identity of objects	a is b, a is not b

Example :

a = 10, b = 3		a & b → 2	# Bitwise
a + b = 13	# Arithmetic	"on" in "python" → True	# Membership
a > b → True	# Comparison	a is b → False	# Identity
a += 5 → a = 15	# Assignment	a is not b → True	# Identity
a > 5 and b < 10 → True	# Logical		

16. Control Flow Statements

- Control flow statements are used to control the execution of a program.
- Python has the following control flow statements.



Example :

1. <u>if statement</u> <pre>x = 10 if x > 5: print("x is greater than 5")</pre>	2. <u>for loop</u> <pre>for i in range(1, 4): print(i) # Output: 1 2 3</pre>	3. <u>while loop</u> <pre>i = 1 while i <= 3: print(i) i += 1 # Output: 1 2 3</pre>
---	---	---

★ Control flow statements help in making programs more powerful and efficient. ★

17. Functions in Python

- A function is a block of code that performs a specific task.
- It helps to reuse code and make programs more modular.

Syntax :

```
def function_name(parameters):
    # code to be executed
    return value
```

Example :

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)      # Output: 8
```

18. Types of Functions

Type	Description	Example
Built-in Function	Predefined functions in Python.	print(), type(), len(), input()
User-defined Function	Functions created by the user.	def my_func(): ...
Anonymous Function	Functions without a name (created using lambda).	lambda x: x * x

19. Modules in Python

- A module is a file containing Python code (functions, variables, classes, etc.).
- It helps to organize code in separate files and reuse it.

Example :

1. Create a file named `mymodule.py`
2. Write some functions in it.
3. Import the module in another program using `import`.

```
# In mymodule.py
def greet(name):
    return "Hello, " + name

# In another file
import mymodule
print(mymodule.greet("Monika"))

# Output: Hello, Monika
```

20. Comments in Python

- Comments are used to explain the code.
- Python supports single-line and multi-line comments.

Single-line Comment	Multi-line Comment
# This is a single-line comment print("Hello") # This prints Hello	""" This is a multi-line comment """ print("Welcome")



Functions, modules and comments make the code organized, readable and easy to maintain.



19. Tuples in Python

- A tuple is an ordered, immutable collection of items.
- Tuples are written using parentheses ().

Method	Description	Example	Output
count(x)	Returns number of occurrences of x	t.count(2)	2
index(x)	Returns index of first occurrence of x	t.index(30)	2
len(t)	Returns length of tuple	len(t)	4
max(t)	Returns maximum value	max(t)	30
min(t)	Returns minimum value	min(t)	10

Example :

```
t = (10, 20, 30, 20)
print(t.count(20)) # 2
print(t.index(30)) # 2
print(len(t)) # 4
print(max(t)) # 30
print(min(t)) # 10
```

20. Sets in Python

- A set is an unordered collection of unique items.
- Sets do not allow duplicate values.

Method	Description	Example	Output
add(x)	Adds an element x to the set	s.add(4)	{1, 2, 3, 4}
remove(x)	Removes element x (raises error if not present)	s.remove(2)	{1, 3}
discard(x)	Removes element x (no error if not present)	s.discard(5)	{1, 3}
pop()	Removes and returns a random element	s.pop()	1 (or 3)
clear()	Removes all elements	s.clear()	set()
len(s)	Returns number of elements	len(s)	3
union(s2)	Returns union of two sets	s.union(s2)	{1, 2, 3, 4, 5}
intersection(s2)	Returns common elements	s.intersection(s2)	{3}
difference(s2)	Returns elements in s not in s2	s.difference(s2)	{1, 2}

Example :

```
t = {1, 2, 3}
s.add(4) # {1, 2, 3, 4}
s.remove(2) # {1, 3, 4}
s.discard(5) # {1, 3, 4}
popped = s.pop() # 1 or 3 or 4 (random)
print(s) # remaining set
s.clear() # set()
```

★ Sets are useful when you need only unique items and fast membership testing. ★

21. Dictionaries in Python

- A dictionary is a collection of key-value pairs.
- Dictionaries are mutable and unordered (before Python 3.7).

Method	Description	Example	Output
get(key)	Returns value for key	d.get('name')	'Monika'
keys()	Returns all keys	d.keys()	dict_keys(['name', 'age', 'city'])
values()	Returns all values	d.values()	dict_values(['Monika', 24, 'Indore'])
items()	Returns key-value pairs	d.items()	dict_items([('name', 'Monika'), ('age', 24), ('city', 'Indore')])
update(d2)	Updates dictionary with d2	d.update({'age': 25})	Updated dictionary
pop(key)	Removes key and returns value	d.pop('city')	'Indore'
popitem()	Removes last inserted item	d.popitem()	('age', 24)
clear()	Removes all items	d.clear()	{}
len(d)	Returns number of items	len(d)	3

Example :

```
d = {'name': 'Monika',
      'age': 24, 'city': 'Indore'}
print(d.get('name')) # Monika
print(d.keys()) # dict_keys(['name', 'age', 'city'])
print(d.values()) # dict_values(['Monika', 24, 'Indore'])
print(d.items()) # dict_items([('name', 'Monika'), ('age', 24), ('city', 'Indore')])
d.update({'age': 25})
print(d) # Updated dict
print(d.pop('city')) # Indore
print(d.popitem()) # ('age', 25)
print(d) # {'name': 'Monika'}
d.clear()
print(d) # {}
```

★ Tuples are immutable, Sets contain unique items and Dictionaries store key-value pairs. All three are important data structures in Python. ★

23. Functions in Python

- A function is a block of code that performs a specific task.
- It improves code reusability, readability and reduces repetition.
- Functions are defined using the `def` keyword.

Syntax :

```
def function_name(parameters):
    statement(s)
    return value # (optional)
```



Quick Note

Functions help in breaking a large program into smaller parts. This makes the program easy to understand and maintain.

Types of Functions

1. Built-in Functions → Provided by Python.
2. User-defined Functions → Created by the programmer.
3. Anonymous (Lambda) Functions → Small, unnamed functions.

Example :

Type	Example Code	Description	Output
1. Built-in Function	<pre>print(len("Python")) print(max(10, 20, 5))</pre>	<code>len()</code> returns length <code>max()</code> returns largest value	6 20
2. User-defined Function	<pre>def add(a, b): sum = a + b return sum print(add(5, 7))</pre>	User-defined function that adds two numbers.	12
3. Lambda Function	<pre>square = lambda x: x * x print(square(6))</pre>	Lambda function to calculate square.	36

Function with Different Types of Arguments

Type of Argument	Example	Explanation
1. Positional Arguments	<pre>def add(a, b): return a + b add(5, 7)</pre>	Values are passed in correct positional order.
2. Keyword Arguments	<pre>add(b = 7, a = 5)</pre>	Values are passed using parameter names.
3. Default Arguments	<pre>def greet(name = "Guest"): print("Hello", name) greet()</pre>	If no value is passed, default value is used.
4. Variable Length Arguments	<pre>def total(*numbers): return sum(numbers) print(total(1, 2, 3, 4))</pre>	Can accept any number of positional arguments.
5. Keyword Variable Arguments	<pre>def info(**data): print(data) info(name = "Monika", age = 21)</pre>	Can accept any number of keyword arguments.

Example Program

```
def multiply(a, b):
    return a * b

x = 6
y = 4
result = multiply(x, y)
print("Result =", result)
```

Output :

Result = 24



Functions make your code modular, reusable and easy to debug.



25. Packages in Python

- A package is a collection of modules.
- Packages help in organizing related modules in a directory hierarchy.
- Packages are imported just like modules.

Quick Note

Packages help in managing large projects by grouping related modules together.

Creating and Using Packages

Step	Explanation
1. Create a directory (package name).	Example: mypackage/ (This folder is a package)
2. Add <code>__init__.py</code> file in the package directory.	It tells Python that this directory is a package.
3. Add modules (.py files) inside the package.	Example: mypackage/mymodule.py
4. Import and use the modules.	Syntax: <code>from package_name import module_name</code> or <code>from package_name.module_name import name</code>

Example

Package Structure:

```
mypackage/
|__ __init__.py
|__ mymodule.py
main.py
```

File: mypackage/mymodule.py

```
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

File: main.py

```
from mypackage import mymodule
print(mymodule.greet("Monika")) # Hello, Monika!
print(mymodule.add(10, 20))    # 30
```

Output:

```
Hello, Monika!
30
```

26. File Handling in Python

- File handling allows us to create, read, write, update and delete files.
- Python provides built-in functions to work with files.

Opening a File

Mode	Description	Operation
'r'	Read mode	Opens file for reading (Default mode)
'w'	Write mode	Opens file for writing. Creates new file or truncates existing file.
'a'	Append mode	Opens file for appending. Creates new file if it does not exist.
'x'	Exclusive creation mode	Creates a new file, returns error if file exists.
'b'	Binary mode	Opens file in binary mode.
't'	Text mode	Opens file in text mode. (Default)
'+'	Update mode	Opens file for both reading and writing.

Common File Methods

<code>read()</code>	Read entire file
<code>read(n)</code>	Read first n characters
<code>readline()</code>	Read one line
<code>readlines()</code>	Read all lines as a list
<code>write(str)</code>	Write string to file
<code>writelines(list)</code>	Write list of lines
<code>close()</code>	Close the file
<code>seek(pos)</code>	Move file pointer to position
<code>tell()</code>	Return current file pointer
<code>truncate()</code>	Truncate the file

Examples

1. Writing to a File

```
f = open("data.txt", "w")
f.write("Hello Python!\n")
f.write("File handling is easy.")
f.close()
```

Output in data.txt:

```
Hello Python!
File handling is easy.
```

2. Reading a File

```
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

Output:

```
Hello Python!
File handling is easy
```

3. Appending to a File

```
f = open("data.txt", "a")
f.write("\nHave a nice day!")
f.close()
```

Output in data.txt:

```
Hello Python!
File handling is easy.
Have a nice day!
```

4. Reading Line by Line

```
f = open("data.txt", "r")
for line in f:
    print(line.strip())
f.close()
```

Output:

```
Hello Python!
File handling is easy.
Have a nice day!
```



File handling is an important skill in programming.
Always close the file after use using `close()` to free resources.



27. Exception Handling in Python

- Exceptions are errors that occur during program execution.
- Exception handling prevents program crash and allows normal flow.
- We use try, except, else, finally blocks.

Quick Note

Good exception handling makes our program robust and user-friendly.

Syntax

```
try:
    # code that may raise exception
except ExceptionType:
    # code to handle exception
else:
    # code to run if no exception
finally:
    # code that always runs
```

Example

```
a = 10
b = 0
try:
    c = a / b
    print("Result =", c)
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Division successful.")
finally:
    print("This block always executes.")
```

Output:

Cannot divide by zero!
This block always executes.

Common Exceptions in Python

Exception	Description	Example
ZeroDivisionError	Raised when division or modulo by zero occurs.	10 / 0
ValueError	Raised when a function gets argument of correct type but inappropriate value.	int("abc")
TypeError	Raised when operation or function is applied to an object of inappropriate type.	"5" * 10
NameError	Raised when a variable or name is not found.	print(x) # x is not defined
IndexError	Raised when index is not found in sequence.	list1 = [1,2,3] print(list1[5])
FileNotFoundError	Raised when file or directory is not found.	open("abc.txt")
ImportError	Raised when module cannot be imported.	import unknownmodule

Tips

- Use specific exceptions instead of generic Exception.
- Always use finally block to release resources.
- Keep try block small and focused.
- Provide meaningful error messages.

28. List Comprehension in Python

- List Comprehension provides a concise way to create lists. Examples
- It is faster and more readable than traditional loops.

Basic Syntax

```
new_list = [expression for item in iterable
            if condition]
```

Nested List Comprehension

```
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
# 3x3 Multiplication Table
Output: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

Example	Output
1. squares = [x*x for x in range(1,6)]	[1, 4, 9, 16, 25]
2. even = [x for x in range(1,11) if x%2==0]	[2, 4, 6, 8, 10]
3. cubes = [x**3 for x in range(1,4)]	[1, 8, 27]
4. names = ["monika", "patel", "bca"] upper = [name.upper() for name in names]	['MONIKA', 'PATEL', 'BCA']
5. nums = [1,2,3,4,5,6] odd_squares = [x*x for x in nums if x%2!=0]	[1, 9, 25]

29. Lambda Function (Anonymous Function)

- Lambda functions are small anonymous functions.
- Defined using lambda keyword. Useful for short operations.

Syntax

```
lambda arguments: expression
```

Examples

Example	Output
1. add = lambda a, b: a + b add(5, 3)	8
2. square = lambda x: x*x square(4)	16
3. even = lambda x: x % 2 == 0 even(10)	True
4. max_num = lambda a, b: a if a>b else b max_num(10, 20)	20

Use in Built-in Functions

```
nums = [1, 5, 2, 9, 3]
# Sort using lambda
nums.sort(key=lambda x: x)
print(nums)
# [1, 2, 3, 5, 9]
# Find max using lambda
maximum = max(nums, key=lambda x: x)
print(maximum)
# 9
```



Exception handling handles errors gracefully.
List comprehension and Lambda functions make code shorter, cleaner and efficient.

